

Rust en Producción

Luca Palmieri y enfoque del libro

El episodio 672 presenta una entrevista a Luca Palmieri, ingeniero backend y arquitecto de software, autor del libro *Zero to Production in Rust*. Ha trabajado en empresas como AWS y actualmente es Ingeniero Principal y Consultor Técnico en MainMatter, además de formar parte activa de la comunidad de Rust desde 2018.

Rust en producción

En producción, Rust no elimina los problemas del software, pero permite gestionarlos de forma más segura gracias a su sistema de tipos, que detecta errores en compilación antes de llegar a producción.

Además, genera binarios auto-contenidos que simplifican el despliegue sin runtimes adicionales. Estos pueden ejecutarse directamente o en Docker con imágenes mínimas. En producción se usa el perfil *Release*, ya que optimiza rendimiento, tamaño y eficiencia.

Optimización: LTO y PGO

Rust incorpora técnicas de optimización como LTO y PGO.

LTO (Link Time Optimization) mejora el rendimiento optimizando entre *crates* en el enlace, aunque aumenta el tiempo de compilación. PGO (Profile Guided Optimization) usa datos reales de ejecución para optimizar las partes más utilizadas del programa. Ambas se aplican cuando se busca el máximo rendimiento.

Aplicación ejemplo

El libro presenta una aplicación tipo newsletter con gestión de suscriptores, envío de correos, verificación de usuarios, autenticación y administración. El objetivo es mostrar el desarrollo completo de una aplicación real: estructura, organización de módulos y conexión con base de datos, con enfoque backend basado en APIs.

Inner Development Loop y estructura

El *Inner Development Loop* describe el ciclo de desarrollo: escribir código, compilar, ejecutar tests y validar resultados. En Rust es clave este ciclo porque cada cambio requiere recompilación.

Para optimizarlo, el proyecto se divide en *crates*, lo que permite recompilar solo partes modificadas. Sin embargo, una fragmentación excesiva dificulta la comunicación entre módulos, por lo que deben definirse límites equilibrados en la arquitectura.

Compilación y Docker

En local Rust usa compilación incremental, pero en Docker se pierde por el sistema de capas, que invalida la caché si cambia el código.

Para optimizarlo se separa la compilación de dependencias y del código, aprovechando Cargo.lock para reutilizar capas. Herramientas como cargo-chef mejoran este proceso y aceleran la construcción de imágenes Docker.

Seguridad en Rust

El principal vector que los atacantes utilizan para lograr ejecución remota de código en lenguajes de sistemas tradicionales son los fallos en la gestión de la memoria. Rust altera radicalmente este panorama gracias a su modelo de propiedad (ownership) y su sistema de tipos. Siempre que el desarrollo evite los bloques de código marcados como unsafe, los cuales permiten la manipulación manual de punteros, el lenguaje garantiza la inmunidad frente a estas vulnerabilidades de memoria.

En Rust el mayor factor de riesgo en cuanto a vulnerabilidades es el uso de librerías externas desconocidas para el desarrollador, ya que pueden contener código unsafe o podrían ser modificadas con fines maliciosos.

Rendimiento, Memoria y la Ausencia de un GC

Una de las mayores ventajas de Rust es su reducido uso de memoria. A diferencia de otros lenguajes, el compilador de Rust sabe cuándo una variable deja de estar en uso y la desasigna instantáneamente. Esto se traduce en una huella de memoria minúscula.

En entornos empresariales de microservicios, migrar aplicaciones de lenguajes como C# a Rust ha demostrado reducir drásticamente la factura de computación en la nube para manejar el mismo volumen de tráfico.

El argumento más fuerte a favor de Rust en sistemas de alta concurrencia reside en la consistencia de su latencia. Para medir la experiencia del usuario, la industria utiliza percentiles: el P50 y el P99 (la latencia experimentada para un % de usuarios). El objetivo es reducir la banda entre el P50 y el P99. Al carecer de GC, la latencia en Rust se mantiene predecible y la banda entre su P50 y P99 es increíblemente estrecha.

Manejo de Errores: Errores como Valores

Rust, inspirado en lenguajes funcionales, rechaza por completo el uso de excepciones. En su lugar, modela los errores como valores a través del enumerado fuertemente tipado Result. El tipo Result tiene dos variantes:

- Ok(valor): Contiene los datos si la operación fue exitosa.
- Err(error): Contiene la información del error si la operación falló.

En este diseño el sistema de tipos te prohíbe acceder al valor sin antes haber definido qué hacer con el error. Estás forzado, por diseño del lenguaje, a lidiar con el fracaso.

Dado que los errores son explícitos y forman parte de la firma de la función, cualquier cambio futuro es detectado por el compilador. Si introduces un nuevo modo de fallo el compilador de Rust detendrá la construcción del proyecto inmediatamente.

Antipatrones

El antipatrón más crítico que debe erradicarse es el uso de interrupciones no recuperables.

Un panic! en Rust equivale a estrellar el programa intencionadamente. En el contexto de una API web, si un usuario envía un payload malformado y el servidor falla al intentar deserializarlo, la aplicación nunca debería colapsar. En su lugar, el error debe propagarse ordenadamente hacia arriba y transformarse en una respuesta HTTP controlada. Dejar un .unwrap() en código de producción es dejar una mina terrestre que tirará el servicio en cuanto se encuentre un caso límite.